

EGOI 2024 Editorial - Infinite Race

Problem author: Nils Gustafsson.

The problem

Anika is one of the N participants (numbered from 0 to $N - 1$) in a race around a circular track. She is participant 0 and starts right after the finish line with all other participants positioned ahead of her on the track. Anika cannot keep track of how many laps she has run, but she remembers when she overtakes someone or when someone overtakes her. What is the minimum number of times she must have crossed the finish line? Nobody moves backwards, and no overtaking happens exactly at the finish line. Furthermore, note that the participants do not necessarily run at a constant speed.

Test group 1: $N = 2$

In this test group there is just one other participant.

First, why is the answer not always 0, in other words how can we be sure that Anika has crossed the finish line? In case she overtakes the other participant twice in a row, since the other participant does not run backwards, Anika must have completed a full lap in between and has therefore crossed the finish line at least once.

It turns out that this is the only interesting case. More specifically, the answer is equal to the number of pairs of consecutive events in the input that both correspond to Anika overtaking the other participant (denoted as 1 in the input format).

It is clear that this is a lower bound on the answer. In order to see how we can achieve exactly this many crossings of the finish line, assume that both Anika and the other participant are running very slowly unless they need to overtake, and carefully consider whether Anika or the other participant is ahead within the lap (in other words, whether we first encounter Anika or the other participant if we go from the finish line along the track).

When Anika overtakes the other participant (event 1), she is now ahead of them, and in case she was also ahead before, we add one more finish line crossing to the answer. When the other participant overtakes Anika (event -1), the other participant is now ahead, so there will be no finish line crossing on the next event 1.

Test group 2: Anika only overtakes

In this test group in all events Anika overtakes other participants.

We can assume the other participants are running so slowly that they are effectively standing still, and simply count how many times did she overtake each of the other participants, and find the maximum of those counts. This number minus one will be the answer.

Test group 3: general case, few events

In this test group all types of overtake events are possible, but the number of events that happen is at most 100.

We need to generalize our solution from test group 1. First, similar to that test group assume that both Anika and the other participants are running very slowly unless they need to overtake. Now each of the other participants can be in one of two states: whether they are ahead or behind Anika within the lap.

Whenever Anika overtakes somebody that was ahead of her within the lap, we just need to change the state of that participant so that they are now behind. We can always make this happen since the other participants can overtake each other in such a way that the participant Anika is going to overtake is the next one on the track.

Whenever Anika overtakes somebody that was already behind of her within the lap, then she must cross the finish line one more time. When this happens, we also need to reset the state of all other participants to being ahead of Anika on the track.

Similar to test group 1, this approach computes a lower bound on the answer, and having the participants only move around as much as is needed for the overtakes we can achieve that bound.

This solution runs in quadratic time since it potentially needs to reset the state of all participants every time.

Test group 4: general case, many events

In this test group all types of overtake events are possible, and the number of events that happen is at most $2 \cdot 10^5$.

We just need to speed up the solution from test group 3 slightly so that resetting the state of all participants is faster. There are several typical approaches to this:

- In addition to storing a boolean to represent the state of each other participant, we also store a vector of indices of participants that are behind Anika. This way we can reset the state of all participants to being ahead of Anika in time proportional to the number of participants that were behind, instead of time proportional to the total number of participants. Since at most one participant switches to be behind Anika at every event, the amortized complexity of this approach is $O(1)$ per event, so the running time is $O(N + Q)$ where Q is the number of events.
- Another option is to store the participants that are behind Anika in a set, and simply clear this set when Anika starts a new lap. The running time is $O(Q)$ if set operations are $O(1)$, or $O(Q \log N)$ if the set operations are $O(\log N)$.
- Yet another option is to store an integer instead of a boolean for each of the other participants: on which lap was the participant behind Anika. This way we do not need to reset anything when Anika starts a new lap, just increment the current lap counter (the answer), so the running time is $O(N + Q)$.

EGOI 2024 Editorial - Bouquet

Problem author: Jasmin Studer.

The problem

There are N tulips numbered from 0 to $N - 1$ growing in a line along the road, in order from left to right. The tulip protection law assigns two integers l_i and r_i to every tulip. In case tulip i is picked, the l_i tulips immediately to the left of tulip i , and the r_i tulips immediately to the right of tulip i cannot be picked. What is the maximum amount of tulips that can be picked?

Test group 1: uniform tulips

In this test group, there exists a constant c such that $l_i = r_i = c$ for all i , that is all the l 's and r 's in the input have the same value.

This means that any two picked tulips must be at least $c + 1$ apart. Thus, we can pick at most one tulip from every $c + 1$ consecutive tulips, and we cannot do better than picking tulips $0, c + 1, 2(c + 1), \dots$.

Therefore, the answer is $\lceil \frac{N}{c+1} \rceil$ (rounded up).

Test group 2: no constraints on the right

In this test group $r_i = 0$ for all i .

We can adapt the solution from test group 1 in the following greedy manner: First, we pick tulip $a_0 = 0$. Then, we pick the next tulip a_1 that can be picked according to the constraints, in other words the smallest a_1 that has $l_{a_1} < a_1$. Then, we pick the smallest a_2 such that $l_{a_2} + a_1 < a_2$, and so on.

To prove that this is optimal, we can use the exchange argument: suppose there is an optimal solution that does not pick a certain tulip a_k from the greedy solution above. Consider the next tulip a'_k that is picked by that solution instead, by the construction of a_k we have $a'_k > a_k$. We can then replace a'_k by a_k in that solution and obtain another valid optimal solution. By doing this repeatedly, we can prove that the greedy solution is optimal.

Test group 3: low N

In this test group, we need to solve the general case, but with $N \leq 1000$.

The greedy strategy is no longer optimal, as can be seen from the second sample. However, it is still true that the constraints are applied locally, on pairs of adjacent tulips. Therefore, if we pick tulips from left to right, at every moment only the location of the last picked tulip and the total number of picked tulips matter for the rest of the process.

This naturally leads to the following dynamic programming approach: we compute dp_i as maximum number of tulips that can be picked from tulips $0, 1, \dots, i$ in such a way that tulip i is definitely picked.

We can use the following formula to compute dp_i in increasing order of i :

$$dp_i = \max(1, \max_{j: j < i, \max(r_j, l_i) < i-j} dp_j + 1)$$

This leads to a $O(N^2)$ solution which is fast enough for this test group.

Test group 4: small l_i, r_i

In this test group $l_i, r_i \leq 2$.

We can notice that any two picked tulips in an optimal solution cannot be 6 or more apart, since in that case we can add another tulip in the middle without violating the constraints, contradicting the fact that the solution is optimal.

Therefore we can only iterate over $j \geq i - 5$ in the above dynamic programming solution, which brings its running time down to $O(N)$.

Test group 5: general case

In this test group we need to solve the general case, and $N \leq 2 \cdot 10^5$.

There are multiple ways to approach this problem, for example by using a segment tree to speed up the $O(N^2)$ dynamic programming solution from test group 3.

Consider the condition that we have on j : $\max(r_j, l_i) < i - j$. Let us split this condition into two: $r_j < i - j$ and $l_i < i - j$, which can be rewritten as $j + r_j < i$ and $j < i - l_i$. If we put the array dp_j into a segment tree, then we can quickly find the maximum of all dp_j where $j < i - l_i$. However, how do we deal with the additional $j + r_j < i$ constraint?

In order to do that, we need to time the updates of the segment tree properly. More precisely, instead of storing the value of dp_j into the segment tree immediately after computing it, we will postpone it until we process $i = j + r_j + 1$. This way when we process the given i our segment tree contains exactly the values we need and we can simply query the maximum over all $j < i - l_i$.

This solution runs in $O(N \log N)$. However, the segment tree was not really necessary to solve this problem.

Instead of storing the values of dp_j in a segment tree, we can store an array b_k : the minimum value of j such that $dp_j \geq k$ for each k . In this case instead of querying the segment tree we can use binary search on this array to find the last value that is less than $i - l_i$, since the array will always be non-decreasing.

When we need to put a new value of dp_j into the array (at the step $i = j + r_j + 1$), it might seem that we need to change b_k for all k from 0 to dp_j . However, since when computing dp_j we have used a previous value from the array dp that is smaller by 1, actually at most one position in the array b_k needs to be changed: we simply need to do $b_{dp_j} = \min(b_{dp_j}, j)$.

This solution also runs in $O(N \log N)$ since it first sorts an array, and then does a binary search in another array N times.

EGOI 2024 Editorial - Team Coding

Problem author: Yann Viegas.

The problem

You are given a rooted tree (rooted at 0) with N nodes. Each node is assigned a programming language (numbered from 1 to K). l_i will denote the programming language assigned to node i . To complete a programming project, you will choose a node u , then you can apply the following operation any number of times:

- pick two nodes v and w having the same depth in the tree
- swap the programming language assigned to nodes v and w

The value of the project is the number of nodes in the subtree of u that share the same programming language as u . When the leader is u , we will call the value of the project $\text{val}(u)$. Also, in the rest of the editorial, $\text{dep}(u)$ will denote the depth of node u and for programming language c , $\text{cnt}(c)$ will denote the number of nodes u such that $l_u = c$.

Find the maximum value of a project that can be achieved and the minimum number of operations needed to achieve it.

We will first discuss how to find the maximum value of a project. The number of operations will then be easy to get.

Test group 1: Chain

In a chain, no two nodes share the same depth so we can not apply any operation. Thus, we fix u as the leader of the project and count the number of $j > i$ such that $l_j = l_i$. This can be implemented in $O(n)$.

Test group 4: $N \leq 2000$

Let us fix u as the leader of the project and compute $\text{val}(u)$. For a given depth d (depths are computed relatively to the root 0, not to u), $\text{atDep}(u, d)$ will denote the number of nodes at depth d in the subtree of u . We will also denote the number of nodes at depth d having programming language c and lying in the subtree of u as $\text{cAtDep}(u, d, c)$. I claim that:

$$\text{val}(u) = \sum_{d=\text{dep}(u)}^N \min(\text{atDep}(u, d), \text{cAtDep}(0, d, l_u))$$

Note that you can limit the sum to the depths that appear in the subtree of u which gives a way to compute $\text{val}(u)$ in $O(\text{size of the subtree of } u)$.

Test group 2: $K \leq 2$

In this subtask there are only 2 different programming languages. Thus, we can fix the color of the leader of the project (let it be c). There are different ways to solve this subtask.

One possible idea is to reduce the number of leaders to consider to fewer candidates as leaders having the same color might share redundant information.

Indeed, if u and v are such that $l_u = l_v$ and u is an ancestor of v , then $\text{val}(u) \geq \text{val}(v)$. Essentially, all the operations you apply when you pick v as a leader will have the same effect when you pick u as a leader. This way, we can pick the node with the smallest depth (let it be u) and process it the same way we solved $N \leq 2000$. After computing $\text{val}(u)$, we do not need to consider nodes lying in the subtree of u anymore. Then, we pick another node u , which is the node with the smallest depth that is not yet marked. . . . The computations are amortised as all the subtrees we will do computations on are disjoint. This gives an $O(n)$ solution once the order of nodes is fixed (which costs $O(n \log(n))$ to precompute).

Another way to solve that subtask would be to use small to large merging. Our dfs function should return a map (can also be a vector or deque depending on implementation details) containing $\min(\text{atDep}(u, d), \text{cAtDep}(0, d, l_u))$ for all d that are actual depths in the subtree of u (otherwise the size of the map would explode). The function should also return $\text{val}(u)$. The details are left as an exercise to the reader (but the code is quite short). Note that this can be implemented in $O(n)$ as height merging results in linear time.

Test group 3: each programming language is assigned to at most 10 nodes

The constraints lead us to fix the color c of the project. Now, there are at most 10 leaders candidate because of the constraints, so we can fix the leader u . To compute $\text{val}(u)$, we can notice that there are at most 10 non zero terms in the sum. Indeed, $\text{cAtDep}(u, d, c) \neq 0$ for every d such that there is at least one node of color c at that depth (and there are at most 10 such nodes). The only thing left is to be able to compute each term of the sum efficiently. To compute $\text{atDep}(u, d)$: for each depth d , sort all nodes by their euler tour "in" time and use binary search to count nodes that lie in the subtree of u . Thus, we can query $\text{atDep}(u, d)$ in $O(\log(n))$. Finally, we can compute $\text{cAtDep}(u, d, c) \neq 0$ naively by iterating through all u such that $l_u = c$.

We can thus process color c in $O(\text{cnt}(c)^2 \log(n))$. Note that it is also possible to do it in $O(\sum_{c=0}^K \text{cnt}(c)^2)$ by processing all the colors at the same time by doing a height merging, similar to what was discussed for $K \leq 2$.

Test group 5: full solution

Using what was previously discussed, we can process a color in $O(\min(N, \text{cnt}(c)^2))$. The total number of operations is going to be $O(N\sqrt{N})$. One way to see that is to notice that for all c such that $\text{cnt}(c) < \sqrt{N}$, we process them in $O(\text{cnt}(c)^2)$. Thus, we use $\sum_{c|\text{cnt}(c) < \sqrt{N}} \text{cnt}(c)^2$ operations to process them. However,

$$\sum_{c|\text{cnt}(c) < \sqrt{N}} \text{cnt}(c)^2 \leq \sum_{c|\text{cnt}(c) < \sqrt{N}} \text{cnt}(c)\sqrt{N} \leq N\sqrt{N}$$

$$\text{as } \text{cnt}(c) < \sqrt{N} \text{ and } \sum_{c|\text{cnt}(c) < \sqrt{N}} \text{cnt}(c) \leq \sum_{c=0}^K \text{cnt}(c) = N.$$

For c such that $\text{cnt}(c) \geq \sqrt{N}$, we run the $O(n)$ algorithm and there are at most $\sqrt{N}$ such c as $(\# \text{ of such } c) \cdot \sqrt{N} \leq \sum_{c|\text{cnt}(c) \geq \sqrt{N}} \text{cnt}(c) \leq N$

Note that $O(N\sqrt{N}\log N)$ solutions also pass (in case we use the log algorithms from the previous sections) and such solutions are also easier to implement.

Finding number of operations

Assuming the leader is u , the number of operations performed at depth d is

$$\min(\text{atDep}(u, d), \text{cAtDep}(0, d, l_u)) - \text{cAtDep}(c, d, l_u)$$

. We can compute this the same way as we computed $\text{val}(u)$.

EGOI 2024 Editorial - Garden Decorations

Problem author: Massimo Cairo

The problem

There is a sequence of N bits. You need to apply the given permutation to this sequence, but you cannot just read and write random elements of the sequence. Instead, first you go from left to right, and must write a new value for each element after reading its current value. Then, you forget everything you saw (your program is restarted), and you go from right to left, and must write a new value for each element after reading its current value. Then, you forget everything you saw again, and go from left to right, and so on. Your goal is to apply the given permutation correctly in at most W steps, and the lower W is, the better. To get any points, you need $W \leq 60$, and for a full score, you need $W \leq 3$.

Test group 1: $N = 2$

In this test group there are only two possible permutations. If the permutation is the identity permutation we just output $W = 0$ and terminate your program as there is nothing to do.

Else the permutation is a swap of two elements. We can solve this in 3 runs. In the first run, we keep the first element as is ($b_{1,0} = b_{0,0}$) and set $b_{1,1} = b_{0,0} \oplus b_{0,1}$. In the second run, we go through the elements in reverse order. We keep the second element as is ($b_{2,1} = b_{1,1}$) and set

$$b_{2,0} = b_{1,0} \oplus b_{1,1} = b_{0,0} \oplus b_{0,0} \oplus b_{0,1} = b_{0,1}.$$

In the third run we again process the elements from the front. We keep $b_{3,0} = b_{2,0} = b_{0,1}$ and set $b_{3,1} = b_{2,0} \oplus b_{2,1} = b_{0,1} \oplus b_{0,0} \oplus b_{0,1}$.

Test group 2: $N \leq 15$

In this test group we will see how we can use the swap that we established in Test group 1 to process a full permutation. Note that we can represent any permutation as a sequence of at most $N - 1$ swaps. For every swap, we need 3 moves, and one more move to get back to start with a forward move. This solves the problem in $4 \cdot (N - 1) \leq 56$ moves.

Going back is actually not necessary, and we can instead start the swap in the back pass. This gives $3 \cdot (N - 1) \leq 42$ moves in total. You can find ideas on reducing W even further in the solutions for test group 5.

Test group 3: Reversal

In this test group the given permutation is the reverse permutation, and therefore it consists of many independent swaps.

Note that we do not have to do swaps one by one. Instead, we can apply exactly the same computation for several independent swaps in the same three passes. Since in this test group all swaps are independent, we can solve it this way with $W = 3$.

Test group 4: Shift to the right

In this test group the permutation is a cyclic shift to the right. We can generalize the approach from test group 1 as follows.

In the first run, we set each element to the sum of this element and all previous elements:

- $b_{1,0} = b_{0,0}$
- $b_{1,1} = b_{0,0} \oplus b_{0,1}$
- $b_{1,2} = b_{0,0} \oplus b_{0,1} \oplus b_{0,2}$
- $\dots$
- $b_{1,N-1} = b_{0,0} \oplus b_{0,1} \oplus \dots \oplus b_{0,N-1}$

,

In the second backward run, we set each element to the sum of this element and the next element. Most of the original terms cancel out, and we get:

- $b_{2,N-1} = b_{1,N-1} = b_{0,0} \oplus b_{0,1} \oplus \dots \oplus b_{0,N-1}$
- $b_{2,N-2} = b_{1,N-1} \oplus b_{1,N-2} = b_{0,N-1}$
- $b_{2,N-3} = b_{1,N-2} \oplus b_{1,N-3} = b_{0,N-2}$
- $\dots$
- $b_{2,1} = b_{0,2}$
- $b_{2,0} = b_{0,1}$

Almost all elements are already correct, we just need to add all other elements to the last element in the third run:

- $b_{3,0} = b_{2,0} = b_{0,1}$
- $b_{3,1} = b_{2,1} = b_{0,2}$
- $\dots$
- $b_{3,N-2} = b_{0,N-1}$
- $b_{3,N-1} = b_{2,0} \oplus b_{2,1} \oplus \dots \oplus b_{2,N-1} = b_{0,0}$

,

Test group 5: Shift to the left

In this test group the permutation is a cyclic shift to the left. We can generalize the approach from test group 1 in a slightly different manner.

In the first run, we keep the first element unchanged, and set each subsequent element to the sum of this element and previous element:

- $b_{1,0} = b_{0,0}$
- $b_{1,1} = b_{0,0} \oplus b_{0,1}$
- $b_{1,2} = b_{0,1} \oplus b_{0,2}$
- $\dots$
- $b_{1,N-1} = b_{0,N-2} \oplus b_{0,N-1}$

,

In the second backward run, we set each element to the sum of this element and all subsequent elements. Most of the original terms cancel out, and we get:

- $b_{2,N-1} = b_{1,N-1} = b_{0,N-2} \oplus b_{0,N-1}$
- $b_{2,N-2} = b_{1,N-1} \oplus b_{1,N-2} = b_{0,N-2} \oplus b_{0,N-1} \oplus b_{0,N-3} \oplus b_{0,N-2} = b_{0,N-1} \oplus b_{0,N-3}$
- $b_{2,N-3} = b_{1,N-1} \oplus b_{1,N-2} \oplus b_{1,N-3} = b_{0,N-1} \oplus b_{0,N-4}$
- $\dots$
- $b_{2,1} = b_{0,N-1} \oplus b_{0,0}$
- $b_{2,0} = b_{0,N-1}$

And finally, in the third forward run, we just add the first element to all other elements to cancel it out:

- $b_{3,0} = b_{2,0} = b_{0,N-1}$
- $b_{3,1} = b_{2,1} \oplus b_{2,0} = b_{0,0}$
- $b_{3,2} = b_{2,2} \oplus b_{2,0} = b_{0,1}$
- $\dots$
- $b_{3,N-1} = b_{0,N-2}$

$W = 6$: 90 points

In test group 3, we have managed to solve the reverse permutation with $W = 3$ thanks to the fact that it consists of independent swaps. There are many more such permutations which are also called *permutations of order 2*.

Moreover, it turns out that every permutation can be represented as a product of two permutations of order 2! Which means that we can just apply those two permutations in order, each using 3 passes, to obtain a solution with $W = 6$.

To prove that this is the case, it suffices to show how to represent a cycle in this manner, since this construction can then be applied to all cycles of a permutation independently.

First, consider a cycle of even length. We start with an identity permutation $(0, 1, \dots, 2k-1)$. Let us swap pairs of adjacent elements first, we get: $(1, 0, 3, 2, \dots, 2k-1, 2k-2)$. Now let us swap pairs of adjacent elements, but starting with position 1, we get: $(1, 3, 0, 5, 2, \dots, 2k-1, 2k-4, 2k-2)$. This permutation is one long cycle: it goes from 0 to 1, then over all odd numbers to $2k-1$, then to $2k-2$, and then back over all even numbers to 0.

Since we obtained some long cycle by applying two permutations of order 2 sequentially, we can then obtain the cycle we need just by appropriately renumbering the elements. One can also check that for a cycle of odd length exactly the same construction works correctly.

$W = 5$: 95 points

In the previous solution, we need to apply two permutations of order 2, and we know how to apply each using 3 passes. It turns out that we can apply the last pass of the first permutation simultaneously with the first pass of the second permutation. Indeed, if we build the 3 passes for each permutation as forward-backward-forward, then the last pass of the first permutation and the first pass of the second permutation are both forward passes. So we can apply both of them at the same time by substituting the expressions needed for the first permutation into the expressions needed for the second permutation.

This improves this solution to $W = 5$.

$W = 3$: Full solution

The full solution relies on the definition of a few parameters. For each element $x \in [n]$ we write $S(x)$ and $T(x)$ for the source and the target in the permutation respectively. We denote by $T^i(x)$ the i th successor of x , to make our arguments easier, we include $T^0(x) = x$ in this definition. Formally

$$T^i(x) = \underbrace{T(T(\dots T(x)))}_{i \text{ times}}.$$

We define $S^i(x)$ accordingly.

We then define $Y(x)$ to be the largest element k such that the path from x to $Y(x)$ never goes to an element smaller than x .

$$Y(x) := T^k(x) \text{ where } k = \max_j \{T^i(x) > x \quad \forall i \in [0..j]\}.$$

Note that if the out edge of x goes to the left (i.e. we have $T(x) < x$), then we set $Y(x) = x$.

Similarly, we define $A(x)$ to be the

$$A(x) := S^k(x) \text{ where } k = \max_j \{S^i(x) > x \quad \forall i \in [0..j]\}.$$

Again, if the in edge of x comes from the left (i.e. we have $S(x) < x$), then we set $A(x) = x$.

Additionally, for every cycle in the permutation the smallest element in the cycle will play a special role. We call this smallest element the zero of the cycle.

Algorithm

First forward Phase For the zeros in the cycles we set $b_{1,x} = b_{0,x}$. For every x that is not the zero in a cycle we set $b_{1,x} = b_{0,x} \oplus b_{0,S(A(x))}$. Note that by the definition of $A(x)$, we have that $S(A(x))$ is guaranteed to be smaller than x and is thus available when we compute this.

Backward Phase We define two sequences of vertices

- $P(x) = p_0, p_1, \dots, p_\ell$ with $p_0 = T(x)$ and $p_{i+1} = T(Y(p_i))$ while $p_i > x$. If $T(x) = x$ then $P = \emptyset$. In particular that means that for p_ℓ we have $T(Y(p_\ell)) \leq x$.
- $Q(x) = q_0, q_1, \dots, q_\ell$ with $q_0 = A(x)$ and $q_{i+1} = T(Y(q_i))$ while $p_i > x$. If $A(x) = x$ then $Q = \emptyset$. In particular that means that for q_ℓ we have $T(Y(q_\ell)) \leq x$.

Note that both definitions guarantee all of the visited elements to be strictly larger than x .

1. (for all x) We set $b_{temp} = b_{1,x} \oplus b_{2,p_0} \oplus \dots \oplus b_{2,p_\ell}$,
2. (if x is the zero in a cycle) We set $b_{2,x} = b_{temp}$,
3. (if x is not the zero in a cycle) We set $b_{2,x} = b_{temp} \oplus b_{2,p_0} \oplus \dots \oplus b_{2,p_\ell}$.

Second forward Phase For the zeros in the cycles we set $b_{3,x} = b_{2,x}$. For every x that is not the zero in a cycle we set $b_{3,x} = b_{2,x} \oplus b_{3,T(Y(x))}$. Note that by the definition of $Y(x)$, we have that $T(Y(x))$ is guaranteed to be smaller than x and is thus available when we compute this.

Correctness

We define two parameters to keep track of the state of our algorithm. $F(x)$ and $B(x)$. These two parameters will denote which two bits we are currently storing in the position x .

In the beginning we set $F(x) = \emptyset$ and $B(x) = x$ for every element. We will do the proof for the different phases separately.

Claim 1. *After the end of the first forward phase we have $F(x) = S(A(x))$ and $B(x) = x$ for all $x \neq 0$.*

By construction of the first phase this is exactly what we do. By definition of $A(x)$ we have that $S(A(x)) < x$ for all $x \neq 1$ so we can always xor it into position x .

Claim 2. *After the end of the backward phase we the following two conditions hold for every x that is not a zero.*

$$(C1) \ B(x) = Y(x),$$

$$(C2) \ F(x) = S(x).$$

Proof. We prove this claim by induction over the elements of the permutation, going from the highest element in a cycle backwards to the lowest that is not a zero. We first show the base case, that is that if x is the largest element in a cycle, then it is already in the desired state.

After the first forward phase we had $B(x) = x$. Note that as the last element does not have any elements to the right, we clearly have that the out edge of this element goes to the left and thus $Y(x) = x$ which proves our statement. By the same argument, we have $A(x) = x$ and thus $F(x) = A(S(x)) = S(x)$.

For the inductive step, fix some x and assume that $B(x') = Y(x')$ and $F(x') = S(x')$ holds for all $x' > x$. We show that this implies that the statement is also true for x in two steps. First, we look at $B(x)$, secondly, we consider $F(x)$.

C1: $B(x) = Y(x)$

We look at the elements we xor into x in the first part of the backwards phase.

If $P = \emptyset$ then $T(x) < x$. In this case we have $Y(x) = x$ and thus $B(x) = x$ with no modification satisfies condition.

Else we modify $B(x)$. At the end of this phase we have

$$\begin{aligned} B(x) &= x \oplus p_0 \oplus p_1 \oplus \dots \oplus p_\ell \\ &= x \oplus \underbrace{S(A(T(x)))}_{F(p_0)} \oplus \underbrace{Y(p_0)}_{B(p_0)} \oplus \underbrace{S(T(Y(p_0)))}_{F(p_1)} \oplus Y(p_1) \oplus \dots \oplus \underbrace{S(T(Y(p_0)))}_{F(p_1)} \oplus Y(p_\ell) \end{aligned}$$

One can observe that, by the way we decided to include the elements in our path, we have $F(p_i) = B(p_{i-1})$ and $F(p_0) = x$ as $T(x) > x$ implies $A(T(x)) = T(x)$. This cancels out all but the $B(p_\ell) = Y(p_\ell)$ part of the chain.

We now show that $Y(p_\ell) = Y(x)$. By definition of the path, p_ℓ is a successor of x such that all elements on the path between x and p_ℓ are larger than x . Note that as p_ℓ is the last vertex on the path we know that $T(Y(p_\ell)) \leq x$. This gives that $Y(p_\ell)$ is the last element that can be reached by following the permutation without crossing x , which by the definition of $Y(x)$ implies that $Y(p_\ell) = Y(x)$ as desired.

C2: $F(x) = S(x)$

The proof for $F(x)$ is very similar to the proof for $B(x)$. We look at the elements we xor into x in the second part of the backwards phase. At the end of this phase we have

$$\begin{aligned} F(x) &= S(A(x)) \oplus q_0 \oplus q_1 \oplus \dots \oplus q_\ell \\ &= S(A(x)) \oplus \underbrace{S(A(x))}_{F(q_0)} \oplus \underbrace{Y(q_0)}_{B(q_0)} \oplus \underbrace{S(T(Y(q_0)))}_{F(q_1)} \oplus Y(q_1) \oplus \dots \oplus \underbrace{S(T(Y(q_0)))}_{F(q_1)} \oplus Y(q_\ell) \\ &= S(A(x)) \oplus F(p_0) \oplus B(p_\ell) \end{aligned}$$

First we will show $F(p_0) = S(A(A(x))) = S(A(x))$. This follows from the fact that $A((A(x))) = A(x)$ as the source of the element needs to be larger than $A(x)$. We now show that $Y(q_\ell) = S(x)$. By definition of the path, q_ℓ is a successor of $A(x)$ such that all elements on the path between $A(x)$ and q_ℓ lie on the path between $A(X)$ and x , which is larger than x by definition of $A(x)$. Note that as q_ℓ is the last vertex on the path we know that $T(Y(q_\ell)) \leq x$. As q_ℓ lies on the path between $A(x)$ and x , which lies to the right of x , the only way for an element to become smaller than x is to pass through x . This implies $T(Y(q_\ell)) = x$ and we are done as this immediately gives $Y(q_\ell) = S(x)$. □

Claim 3. *After the end of the backwards phase we have $F(x) = S(x)$ and $B(x) = \emptyset$ for all zeros in the permutation.*

Proof. Note that for the zeros the backwards phase is slightly different. Again, we get

$$\begin{aligned} B(x) &= x \oplus p_0 \oplus p_1 \oplus \dots \oplus p_\ell \\ &= 0 \oplus \underbrace{S(T(0))}_{F(p_0)} \oplus \underbrace{Y(p_0)}_{B(p_0)} \oplus \underbrace{S(T(Y(p_0)))}_{F(p_1)} \oplus Y(p_1) \oplus \dots \oplus \underbrace{S(T(Y(p_0)))}_{F(p_1)} \oplus Y(p_\ell) \\ &= Y(p_\ell) \end{aligned}$$

So all we need to show is $Y(p_\ell) = S(x)$.

By definition, $Y(p_\ell)$ has an out edge going to the left that ends in a position $i \leq x$. This means that $Y(p_\ell)$ is the last vertex in the path of successors of x . In particular, this implies $T(Y(p_\ell)) = x$ as x is the minimum element of the permutation appearing in this cycle and thus $Y(p_\ell) = S(x)$ as desired. $\square$

Claim 4. *After the second forward phase we have $F(x) = S(x)$ and $B(x) = \emptyset$ for all x .*

Proof. We again prove this statement by induction over the elements, this time starting from the front. For the zeros we already have the desired state from Claim 3 so there is nothing to prove.

For x that are not zeros in a cycle, assume that we have processed the elements up to x until now. For all $x' < x$ we get by the induction hypothesis $F(x') = S(x')$ and $B(x') = \emptyset$. That means that when we xor $T(Y(x))$ into x it has the value $S(T(Y(x))) = Y(x)$. This cancels out with $B(x)$ and we get $F(x) = S(x)$ and $B(x) = \emptyset$ for all x . $\square$